Exercise 18 — Contextual Learning with FastAPI

Exercise Description

Learn FastAPI through the lens of frameworks you already know: build a framework "translation table" (Part 1), summarise FastAPI's design philosophy (Part 2), implement JWT authentication (Part 3), and map your existing mental model to FastAPI (Part 4).

Files: - `app.py` — JWT authentication demo (OAuth2 password flow + bearer tokens + dependency-injected auth). - `test_app.py` — 6 tests, all passing. - Dependencies: `fastapi`, `pyjwt`, `python-multipart` (form parsing for the token endpoint). *Password hashing uses stdlib PBKDF2 to avoid a native bcrypt build; the pattern matches passlib.*

Part 1 — Framework Translation Table

Concept I know (Flask/Django)	FastAPI equivalent
Flask route `@app.route`	Path operation `@app.get/post/...`
Flask Blueprint	`APIRouter` (mount with `app.include_router`)
Flask `request.get_json()` + manual validation	Pydantic model as a typed parameter (auto-validated)
Django Forms / DRF Serializers	Pydantic models (`BaseModel`)
Django middleware	Starlette middleware **or** dependencies (`Depends`) for cross-cutting logic
DRF permissions / `login_required`	A dependency like `get_current_user` injected into the path operation
Manual OpenAPI/Swagger setup	Automatic `/docs` and `/redoc`

Part 2 — FastAPI Design Philosophy

- **Why Pydantic (not a bespoke validator)?** Reuse a battle-tested, type-hint-native validation library so request parsing, validation, serialisation, and schema generation all come from one source of truth (the model).
- **Why automatic docs?** Because the same type hints that validate also describe the API, FastAPI can emit an always-accurate OpenAPI schema — solving the perennial "docs drift from code" problem.
- **Why heavy type hints?** They drive validation, editor autocompletion, and docs simultaneously — one annotation, three payoffs.
- **Why async-first?** Built on ASGI/Starlette, it handles high-concurrency I/O (DB, network) without the sync bottleneck of classic WSGI Flask.

Implication for structure: model your data as Pydantic schemas first, push cross-cutting concerns (auth, DB sessions) into dependencies, and let types do the validating.

Part 3 — JWT Authentication (implemented)

Flow: `POST /token` validates credentials (`authenticate_user`) and returns a signed JWT (`create_access_token`, HS256, 30-min expiry). Protected endpoints declare `current_user: User = Depends(get_current_active_user)`; that dependency extracts the bearer token via `OAuth2PasswordBearer`, decodes/validates the JWT, loads the user, and rejects invalid/expired tokens with **401**.

Test run:

```
6 passed in 0.83s
```

Covers: successful login → token; wrong password → 401; protected endpoint without token → 401; with a valid token → 200 (correct user); garbage token → 401; user-scoped items.

How this differs from other frameworks: in Flask/DRF you'd wire a decorator or middleware and pull the user off `request`; in FastAPI auth is just a **dependency** you inject, so it's explicit in each endpoint's signature and appears in the OpenAPI docs automatically.

Part 4 — Mental Model Mapping

```
Django/Flask                    FastAPI
--------------------------------------------------
URL routing        →  path operation decorators
views/controllers  →  path operation functions
models/serializers →  Pydantic models (validation + serialisation)
middleware         →  Starlette middleware  +  Depends() for per-route concerns
auth decorators    →  dependency functions (get_current_user)
manual API docs    →  automatic OpenAPI (/docs)
```

The biggest shift: **dependency injection replaces middleware/decorators** for most cross-cutting logic, and **types replace hand-written validation and docs**.

Reflection Questions

1. **How does FastAPI's auth compare to what I've used?** It's more explicit — auth is a declared dependency in the signature, not hidden middleware — and it self-documents.
2. **What advantage does DI give for auth?** Composable, testable, reusable: `get_current_active_user` builds on `get_current_user`, and either can be overridden in tests.

3. **How does type hinting make security clearer?** The endpoint literally states it needs a `User`; the wiring that produces it is one line and visible.
4. **What patterns from other frameworks did I recognise?** The OAuth2 password flow and bearer-token pattern are the same concepts as DRF's TokenAuthentication, just expressed through dependencies.

Submission for Exercise 18 — Contextual Learning with FastAPI

1. Framework translation table: Flask/Django/DRF concepts mapped to FastAPI equivalents (routes, Blueprints→routers, serializers→Pydantic, middleware/permissions→dependencies, manual→automatic docs).

2. Design-philosophy summary: why Pydantic, automatic docs, pervasive type hints, and async-first — and how each shapes app structure.

3. JWT authentication implementation: `app.py` — OAuth2 password flow, HS256 JWTs with expiry, dependency-injected `get_current_user` / `get_current_active_user`, verified by `test_app.py` (**6 tests passing**); reflection on how it differs from middleware-based auth.

4. Mental-model mapping: the routing/views/models/middleware/auth/docs table above, with the key insight that DI + types replace middleware/decorators and hand-written validation/docs.